

prog2 lab 2025

programma in piu' file sorgenti

Organizzazione in piu' sorgenti

- Come visto a lezione, in tutti i progetti il codice e' suddiviso in files separati, di due tipi:
- file da includere in altri, `<nome.h>` che contengono i prototipi
- file sorgenti `<nome.c>` che contengono le definizioni
- come regola `<nome.c>` include sempre `<nome.h>` per verifica
- i file `<nome.h>` contengono gli elementi da condividere tra tutti i sorgenti:
 - dichiarazioni di costanti (`#define...`)
 - definizioni di tipi
 - dichiarazione di funzioni (prototipi)
 -

le Interfacce

- I file c <nome.h> sono un esempio di Interfaccia
- quando in un programma usiamo un componente esterno dobbiamo fornire al compilatore le informazioni per connetterlo correttamente
- i componenti esterni sono stati precompilati rispettando esattamente le definizioni dello stesso file <nome.h> che includiamo nel nostro sorgente
- il linker non deve quindi far altro che connettere i vari componenti, senza ricompilare

le Interfacce

- i componenti cambiano col tempo ma se le interfacce non cambiano vuol dire che e' un cambiamento interno che non modifica quello esterno
- Quando si creano librerie potenzialmente molto usabili e' fondamentale progettare bene l'interfaccia:
 - deve essere il piu' semplice e minimale possibile
 - non deve mai fare assunzioni sull'implementazione interna del modulo
 - questo per permettere agli implementatori la massima liberta' di evoluzione

Attenzione agli include....

- in un programma ogni simbolo deve essere definito (codice di una funzione, area di memoria per un dato) una sola volta, ma il suo nome (e le proprietà di base) devono essere disponibili ovunque tramite `#include <miofile.h>`
- i file `.h` non devono in generale contenere **definizioni** di variabili o funzioni `{}`
- altrimenti **se** includete il `<nome.h>` in **piu'** sorgenti il linker produrrà errore di **multiple** definizioni.
- capita in progetti piu' complessi che per funzionare correttamente, gli include siano annidati e che un singolo sorgente includa indirettamente piu' volte lo stesso `<nome.h>`

Esempio

- guardiamo la cartella L1_1/esempio :
abbiamo lib1.h lib2.h, dichiarazioni.h, prova.c , prova1.c
- prova.c vuole usare <lib1.h> e <lib2.h>
- sia lib1.h che lib2.h a loro volta includono <dichiarazioni.h> per poter funzionare
- proviamo a scommentare e commentare e vediamo che dice gcc
- in sostanza se inizializziamo con '=' o con {}, abbiamo una definizione, che puo' apparire solo **UNA** volta
- inoltre, la semplice dichiarazione "int a", se appare in piu' sorgenti da' errore di ridefinizione (in sostanza ogni .c riserva memoria per 'int a')

Che fare?

esiste un costrutto speciale del preprocessor fatto così:

```
#ifndef DICHIARAZIONI_H  
#define DICHIARAZIONI_H  
<normale contenuto di dichiarazioni.h>  
#endif
```

se il simbolo non è definito , lo
definisce ed entra, se no salta tutto
perché è già definito



- si aggiunge a tutti i file .h per proteggere il programmatore da queste situazioni.
- In alcuni casi, come visto, reincludere un file non crea problemi, ma non è comunque una buona pratica.

annidamento di include

- Ovviamente non bisogna farsi prendere la mano
- Un conto e' creare una struttura semplice da usare e manutenibile, un conto e' sovraingegnerizzare e creare strutture di include annidate di difficile comprensione

Esercizio

- Abbiamo 3 sorgenti `pot.c` e `fibon.c` `main.c` che realizzano funzioni (ad esempio) ed alcune dichiarazioni.
- se compiliamo ed eseguiamo, riceviamo dei warning ma il programma “funziona”.

riorganizzate il codice creando dei `.h` da includere in modo che:

- 1) non ci siano warning di dichiarazioni implicite o di nessun altro tipo
- 2) non ci siano dichiarazioni duplicate
- 3) si possano usare assieme o indipendentemente tutti i sorgenti
→ potreste dover coordinare/aggiustare le definizioni

Compilare con script
anziche' scrivere ogni volta:

```
gcc *.c  
a.out
```

creiamo un piccolo script che lancia a.out solo se la
compilazione non ha dato alcun warning od errore

vedi DO

Cartelle di un progetto

Una organizzazione semplice e' la seguente:

- una radice con diverse sottocartelle:
 - ogni sottocartella contiene file raggruppati a progetto
 - oppure contiene gruppi di funzioni di utilita' (esempio tempi.c e tempi.h e tempi.o)

lo script

- compila includendo i pathname di tutte le cartelle sorelle necessarie
- linka tutti <pathname>/<NOME>.o delle cartelle sorelle necessari

Cartelle di un progetto

attenzione che se modificate i sorgenti delle altre cartelle dovete andare in quella cartella e ricompilare!

esistono dei programmi appositi che tengono traccia di tutto questo e se necessario ricompilano tutto cio' che serve ma per questo corso non servono

Compilare con script (2)

lo script CREA:

- assume che il programma sia organizzato in piu' cartelle
- in tutte le cartelle ci sono degli oggetti (.o) con i relativi (.h)
- gcc usa la opzione -I coi pathname delle varie cartelle per trovare gli include.
- gcc linka poi tutti i .o della cartella corrente piu' tutti i .o (col pathname) che vi servono dalle altre cartelle